

Lesson 2: Understanding Embeddings

Embeddings are the foundation of RAG systems. This lesson will take you from "what's an embedding?" to understanding how they enable semantic search.

What Are Embeddings?

An embedding is a numerical representation of text (or other data) as a vector in high-dimensional space.

Think of it as translating words into numbers in a way that captures their meaning.

Simple Analogy

Imagine representing fruits by their properties:

```
# 2D embedding: [sweetness, size]
apple  = [7, 5]    # Pretty sweet, medium size
banana = [8, 6]    # Very sweet, large
lemon  = [2, 4]    # Not sweet, medium-small
```

Now you can compute which fruits are similar: - Apples and bananas are close (both sweet, similar size)
- Lemons are far from both (not sweet)

Real embeddings work the same way, but with ~1536 dimensions instead of 2!

Why Do We Need Embeddings?

Traditional keyword search has limitations:

Example: Keyword Search Fails

```
Document: "The Eiffel Tower is a famous landmark in Paris."  
Query: "monuments in France"
```

Keyword match: ■ No shared words → No match!

But semantically: ■ The document IS about a monument in France

Embeddings Capture Meaning

```
# Convert to embeddings (simplified visualization)  
doc_embedding = [0.8, 0.2, 0.9, ...] # Captures "landmark", "Paris", "France"  
query_embedding = [0.7, 0.3, 0.8, ...] # Captures "monument", "France"  
  
# They're similar even with different words!  
similarity = cosine_similarity(doc_embedding, query_embedding)  
# Result: 0.95 (very similar!)
```

How Embeddings Work

The Embedding Model

An embedding model is a neural network trained to convert text into vectors:

```
Text → Embedding Model → Vector
```

Example:

```
text = "The cat sat on the mat"
embedding = embedding_model.encode(text)
print(embedding.shape) # Output: (1536,)
print(embedding[:5]) # First 5 values: [0.023, -0.156, 0.089, 0.234, -0.067]
```

Key Properties

1. Same input = Same output

"hello" always produces the same embedding

Similar meaning = Similar vectors

4. "car" and "automobile" have very similar embeddings

"car" and "pizza" have very different embeddings

High-dimensional space captures nuance

7. 1536 dimensions can represent complex semantic relationships

8. Different dimensions capture different aspects of meaning

Semantic Similarity

Measuring Similarity

The most common metric is **cosine similarity**:

```
import numpy as np

def cosine_similarity(vec1, vec2):
    """Compute cosine similarity between two vectors"""
    dot_product = np.dot(vec1, vec2)
    norm1 = np.linalg.norm(vec1)
    norm2 = np.linalg.norm(vec2)
    return dot_product / (norm1 * norm2)
```

Cosine similarity ranges from -1 to 1: - **1.0**: Identical meaning - **0.0**: Unrelated - **-1.0**: Opposite meaning (rare in practice)

Example: Word Similarities

```
# Hypothetical similarities
similarity("king", "queen")      # 0.85 - very related
similarity("king", "monarch")    # 0.90 - synonyms
similarity("king", "apple")      # 0.05 - unrelated
similarity("happy", "sad")       # 0.20 - related concept, opposite sentiment
similarity("happy", "joyful")    # 0.95 - synonyms
```

Visualizing Embeddings

While we can't visualize 1536 dimensions, we can project to 2D:

```
happy •
      joyful •

sad •
      sorrowful •

car •
      automobile •
      vehicle •
```

Notice: - Synonyms cluster together - Related concepts are nearby - Unrelated concepts are far apart

Embedding Models

Popular Embedding Models

Model	Dimensions	Provider	Best For
<code>text-embedding-3-small</code>	1536	OpenAI	General purpose, fast
<code>text-embedding-3-large</code>	3072	OpenAI	Higher quality
<code>text-embedding-ada-002</code>	1536	OpenAI	Legacy, still good
<code>all-MiniLM-L6-v2</code>	384	Sentence Transformers	Free, fast, local
<code>all-mpnet-base-v2</code>	768	Sentence Transformers	Free, better quality
<code>bge-large-en</code>	1024	BAAI	High quality, free

Choosing an Embedding Model

Factors to consider:

1. **Quality:** How well does it capture semantic meaning?
2. **Speed:** How fast can it generate embeddings?
3. **Cost:** API fees vs. running locally
4. **Dimensions:** More dimensions = more nuance (but slower search)

5. **Language:** Does it support your language(s)?

Recommendations:

- **Starting out:** `all-MiniLM-L6-v2` (free, good quality, easy)
- **Best quality:** `text-embedding-3-large` or `bge-large-en`
- **Production:** Depends on scale and budget
- **Multilingual:** Models like `paraphrase-multilingual-mpnet-base-v2`

Hands-On: Creating Embeddings

Using Sentence Transformers (Free, Local)

```
from sentence_transformers import SentenceTransformer
# Load model (downloads on first use)
model = SentenceTransformer('all-MiniLM-L6-v2')
# Create embeddings
texts = [
    "The quick brown fox jumps over the lazy dog",
    "A fast auburn fox leaps over a sleepy canine",
    "Python is a programming language",
]
embeddings = model.encode(texts)
print(f"Shape: {embeddings.shape}") # (3, 384)
# Compute similarities
from sklearn.metrics.pairwise import cosine_similarity
similarities = cosine_similarity(embeddings)
print(similarities)
# [[1.00, 0.75, 0.15], # First sentence
#  [0.75, 1.00, 0.12], # Second (similar to first!)
```

```
# [0.15, 0.12, 1.00]] # Third (different topic)
```

Using OpenAI Embeddings (Paid, API)

```
from openai import OpenAI

client = OpenAI(api_key="your-api-key")

# Create embedding
response = client.embeddings.create(
    model="text-embedding-3-small",
    input="The quick brown fox jumps over the lazy dog"
)

embedding = response.data[0].embedding
print(f"Length: {len(embedding)}") # 1536
print(f"First 5 values: {embedding[:5]}")
```

Key Concepts Explained

1. Vector Space

Embeddings exist in a **vector space** - a mathematical space where:

- Each dimension represents some learned feature
- Points close together are semantically similar
- Distance/angle between points indicates similarity

2. Dimensionality

More dimensions allow capturing more nuance:

- **384 dimensions**: Good for many tasks
- **768 dimensions**: Better quality
- **1536 dimensions**: High quality, standard for OpenAI
- **3072 dimensions**: Highest quality, slower

Trade-off: Higher dimensions = better quality but slower search

3. Normalization

Embeddings are often normalized (length = 1):

```
import numpy as np

def normalize(vector):
    return vector / np.linalg.norm(vector)
```

This makes cosine similarity equivalent to dot product (faster computation).

4. Context Window

Embedding models have input length limits: - Most models: 512 tokens (~400 words) - Longer models: 8192 tokens (~6000 words)

Solution for long documents: Split into chunks (covered in Lesson 3)

Understanding Similarity Search

When you query a RAG system:

```
# 1. Embed the query
query = "What is the capital of France?"
query_embedding = model.encode(query)

# 2. Compare with all documents
document_embeddings = [embed1, embed2, embed3, ...]
similarities = [
    cosine_similarity(query_embedding, doc_emb)
    for doc_emb in document_embeddings
]

# 3. Return most similar
top_k = 3
```

```
top_indices = np.argsort(similarities)[-top_k:]  
top_documents = [documents[i] for i in top_indices]
```

This is the core of **semantic search**!

Common Pitfalls and Solutions

Pitfall 1: Wrong Model for Task

■ Using a generic model for specialized domain (medical, legal) ■ Use domain-specific embeddings or fine-tuned models

Pitfall 2: Inconsistent Embeddings

■ Using different models for indexing and querying ■ Always use the same model for both

Pitfall 3: Too Long Input

■ Passing 10,000-word document to model with 512 token limit ■ Chunk documents before embedding

Pitfall 4: Not Storing Embeddings

■ Re-computing embeddings every query ■ Compute once, store in vector database

Practical Example: Semantic Search

Let's build a simple semantic search:

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Sample documents
documents = [
    "Paris is the capital of France",
    "The Eiffel Tower is in Paris",
    "Python is a programming language",
    "Machine learning is a subset of AI",
    "The Louvre museum is in France",
]

# Load embedding model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Create document embeddings (done once)
doc_embeddings = model.encode(documents)

def search(query, top_k=3):
    """Semantic search over documents"""
    # Embed query
    query_embedding = model.encode([query])

    # Compute similarities
    similarities = cosine_similarity(query_embedding, doc_embeddings)[0]

    # Get top-k results
    top_indices = np.argsort(similarities)[-top_k:][::-1]

    # Return results
    results = []
    for idx in top_indices:
        results.append({
            'document': documents[idx],
            'similarity': similarities[idx]
        })
    return results

# Test it!
results = search("What museums are in France?")
for r in results:
    print(f"{r['similarity']:.3f}: {r['document']}")

# Output:
# 0.524: The Louvre museum is in France
# 0.398: Paris is the capital of France
# 0.312: The Eiffel Tower is in Paris
```

Notice: It found the Louvre even though the query didn't contain "Louvre"!

What Makes Good Embeddings?

Good embeddings: 1. ■ **Capture semantic meaning** (not just keywords) 2. ■ **Are consistent** (same input → same output) 3. ■ **Cluster related concepts** (similar things are nearby) 4. ■ **Separate unrelated concepts** (different things are far apart) 5. ■ **Work for your domain** (if specialized, use domain-specific models)

Advanced Topics (Preview)

We'll explore these in later lessons:

- **Hybrid search:** Combining embeddings with keyword search
- **Re-ranking:** Using embeddings to re-order results
- **Fine-tuning embeddings:** Training for your specific domain
- **Multi-modal embeddings:** Text + images
- **Approximate nearest neighbor search:** Fast search at scale

What You've Learned

- Embeddings are numerical representations that capture meaning
- Semantic similarity enables finding related concepts
- Different embedding models have different trade-offs
- Cosine similarity measures how related two embeddings are
- You can build semantic search with just embeddings

Practice Exercises

1. **Try it yourself:** Run the semantic search example above with your own documents and queries
2. **Compare models:** Try both `all-MiniLM-L6-v2` and `all-mpnet-base-v2` - notice quality differences?
3. **Experiment:** What happens when you search for opposite concepts? ("happy" in sad documents)

Next Steps

In [Lesson 3: Vector Databases & Retrieval](#), you'll learn: - How to store millions of embeddings efficiently - Fast similarity search algorithms - Choosing and using vector databases - Retrieval strategies and optimization

Further Reading

- [Sentence Transformers Documentation](#)
 - [OpenAI Embeddings Guide](#)
 - [Word2Vec Paper \(foundational\)](#)
 - [BERT Paper \(modern embeddings\)](#)
-

